МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федерального государственного бюджетного образовательного учреждения
высшего образования

«РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ Г.В.ПЛЕХАНОВА»

Техникум Пермского института (филиала)

Сопроводительная записка по дисциплине

«Поддержка и тестирование программных модулей»

по теме:

«Создание и запуск модульных тестов для управляемого кода»

Руководитель

_____/Берестов Д.Б./

«___» _____2026

Исполнитель

_____/Ворошнин И.О./

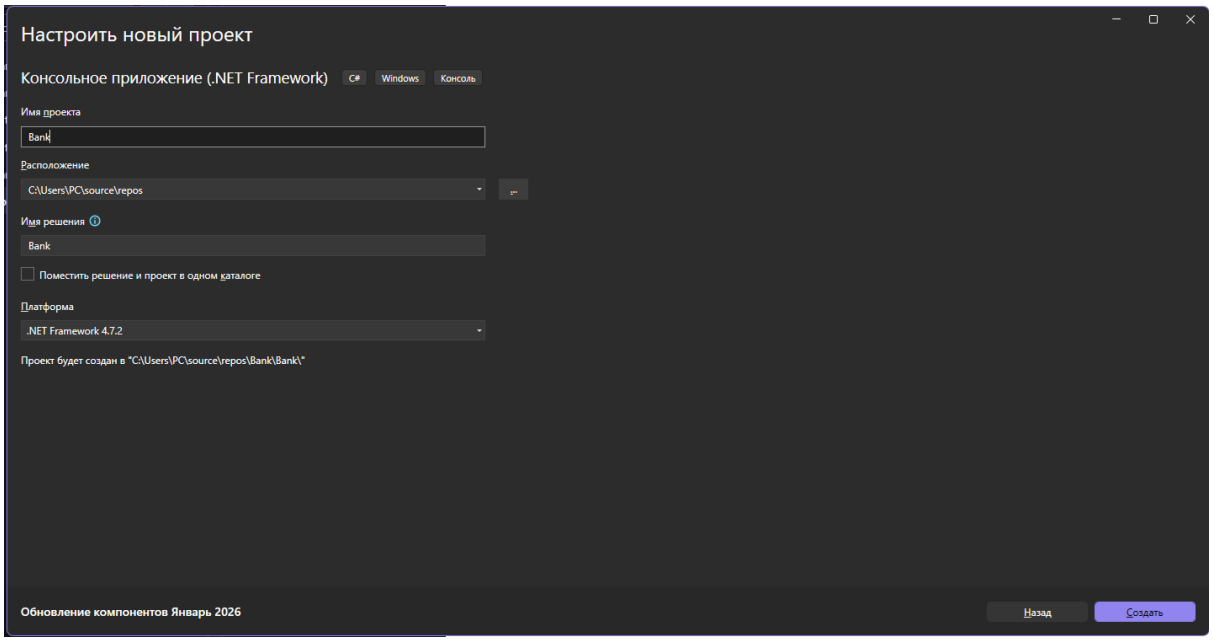
«___» _____2026 г.

Пермь, 2026 г.

Создание проекта для тестирования 3
Создание проекта модульного теста 5
Создание тестового класса 6
Сборка и запуск теста 8
Создание и запуск новых методов теста 9
Рефакторинг тестируемого кода 9
Рефакторинг тестовых методов 10
Вывод 11

Создание проекта для тестирования

Создание проекта (рис.1)



(Рис. 1)

Заменил содержимое файла Program.cs, кодом который оперделяет класс BankAccount:

Также переименовал Program.cs в BankAccount.cs
(рис.2)

BankAccount.cs*

Bank | BankAccountNS.BankAccount

```
using System;
namespace BankAccountNS
{
    /// <summary>
    /// Bank account demo class.
    /// </summary>
    Ссылок: 4
    public class BankAccount
    {
        private readonly string m_customerName;
        private double m_balance;
        Ссылок: 0
        private BankAccount() { }
        Ссылок: 1
        public BankAccount(string customerName, double balance)
        {
            m_customerName = customerName;
            m_balance = balance;
        }
        Ссылок: 0
        public string CustomerName
        {
            get { return m_customerName; }
        }
        Ссылок: 1
        public double Balance
        {
            get { return m_balance; }
        }
        Ссылок: 1
        public void Debit(double amount)
        {
            if (amount > m_balance)
            {
                throw new ArgumentOutOfRangeException("amount");
            }
            if (amount < 0)
            {
                throw new ArgumentOutOfRangeException("amount");
            }
            m_balance += amount; // intentionally incorrect code
        }
        Ссылок: 1
        public void Credit(double amount)
        {
            if (amount < 0)
            {
                throw new ArgumentOutOfRangeException("amount");
            }
            m_balance += amount;
        }
        Ссылок: 0
        public static void Main()
        {
            BankAccount ba = new BankAccount("Mr. Bryan Walton", 11.99);

            ba.Credit(5.77);
            ba.Debit(11.22);
            Console.WriteLine("Current balance is ${0}", ba.Balance);
        }
    }
```

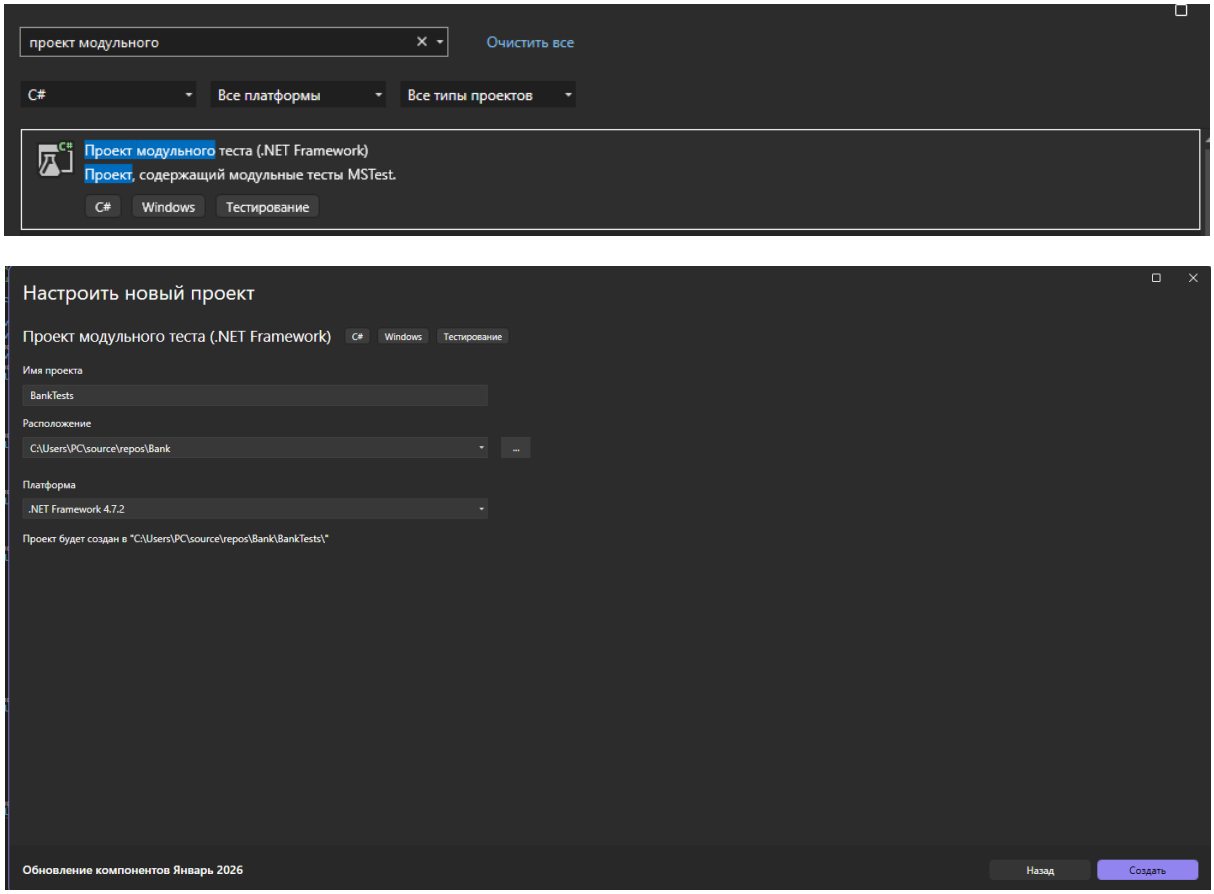
83 % Проблемы не найдены.

(Рис. 2)

В меню сборка построил решение решение (клавиши CTRL + SHIFT + B)

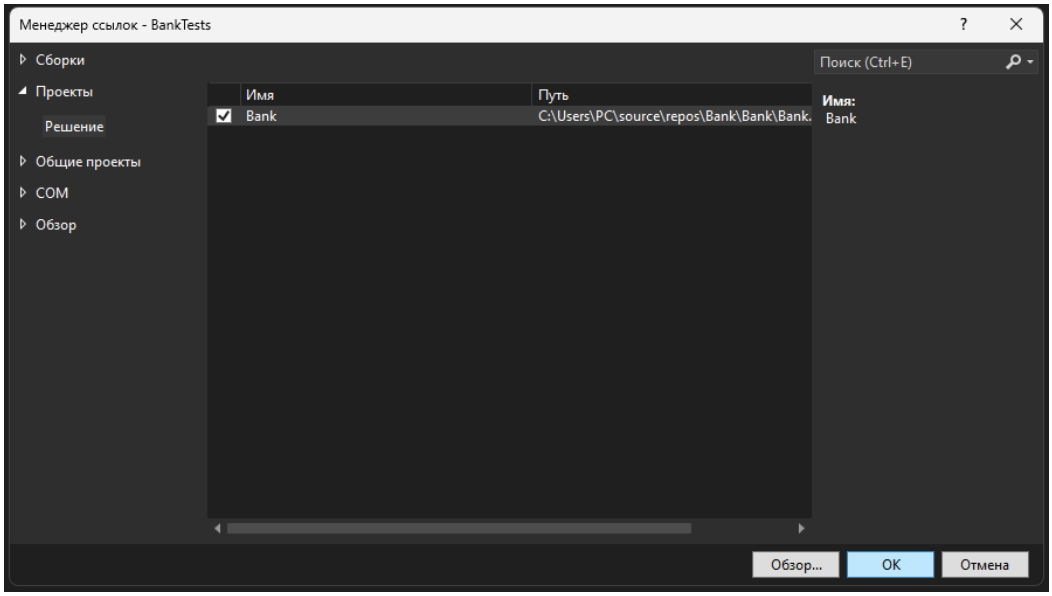
Создание проекта модульного теста

Создаем проект модульного теста (рис.3)



(Рис. 3)

В проекте BankTests добавляю ссылку на проект Банк. (рис.4)



(Рис. 4)

Создание тестового класса

Переименовывал файл UnitTest1.cs в BankAccountTests.cs.

Файл BankAccountTests.cs теперь имеет следующий код: (рис.5)

```
BankAccountTests.cs*   BankAccount.cs
BankTests
1   using Microsoft.VisualStudio.TestTools.UnitTesting;
2   using System;
3
4   namespace BankTests
5   {
6       [TestClass]
        Ссылок: 0
7       public class BankAccountTests
8       {
9           [TestMethod]
            Ссылок: 0
10          public void TestMethod1()
11          {
12          }
13      }
14  }
15
```

(Рис. 5)

Добавил оператор using BankAccountNS; (рис.6)

```
using Microsoft.VisualStudio.TestTools.UnitTesting;
using System;
using BankAccountNS;

namespace BankTests
{
```

(Рис. 6)

Создание метода теста

Первый тест проверяет, снимается ли со счета нужная сумма при допустимом размере кредита (со значением меньшим, чем баланс счета, и большим, чем ноль).

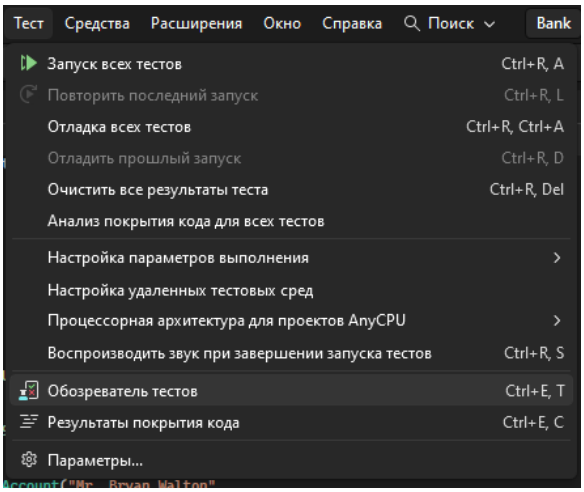
Добавляем предложенный метод в класс BankAccountTest (рис.7)

```
using Microsoft.VisualStudio.TestTools.UnitTesting;
using System;
using BankAccountNS;

namespace BankTests
{
    [TestClass]
    Ссылок: 0
    public class BankAccountTests
    {
        [TestMethod]
        Ссылок: 0
        public void Debit_WithValidAmount_UpdatesBalance()
        {
            // Arrange
            double beginningBalance = 11.99;
            double debitAmount = 4.55;
            double expected = 7.44;
            BankAccount account = new BankAccount("Mr. Bryan Walton",
        beginningBalance);
            // Act
            account.Debit(debitAmount);
            // Assert
            double actual = account.Balance;
            Assert.AreEqual(expected, actual, 0.001, "Account not debited correctly");
        }

    }
}
```

(Рис. 7)

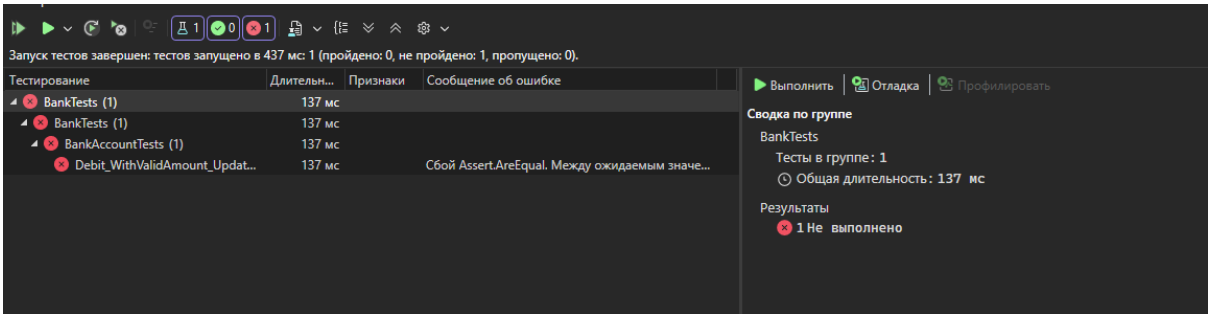
Сборка и запуск теста

В меню сборка построил решение решение (клавиши CTRL + SHIFT + B) Открыл обозреватель тестов, выбрав Тест >Обозреватель тестов в верхней строке меню (клавиши CTRL + E, T) (рис.8)



(Рис.8)

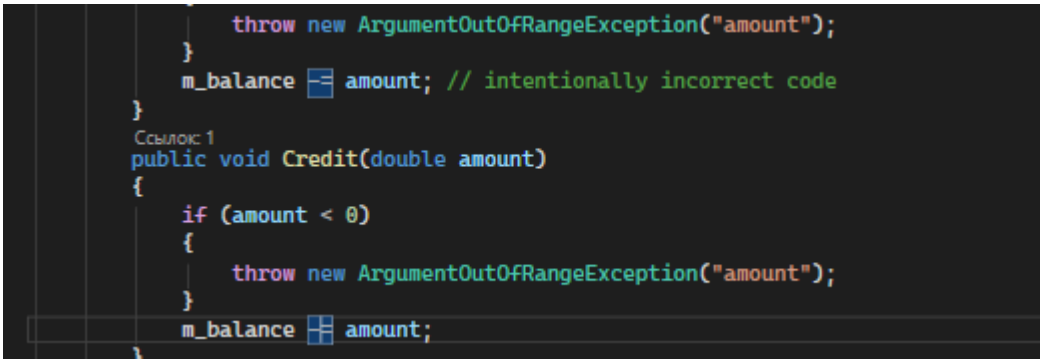
Выполнить все тесты (рис.9)



(Рис. 9)

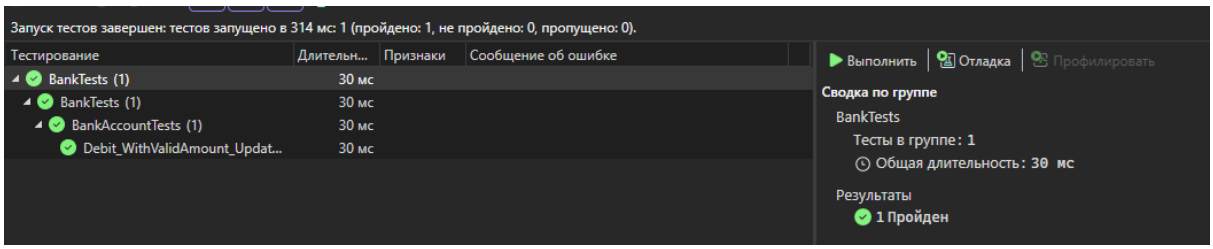
Чтобы исправить эту ошибку, в файле BankAccount.cs заменяем строку:

m_balance += amount; на *m_balance -= amount;* (рис.10)



(Рис. 10)

Повторно запускаю тест. (рис.11)



(Рис. 11)

Создание и запуск новых методов теста

Создаю метод теста для проверки правильного поведения в случае, когда сумма по дебету меньше нуля (рис.12)

```
[TestMethod]
Ссылок: 0
public void Debit_WhenAmountIsLessThanZero_ShouldThrowArgumentOutOfRange()
{
    // Arrange
    double beginningBalance = 11.99;
    double debitAmount = -100.00;
    BankAccount account = new BankAccount("Mr. Bryan Walton",beginningBalance);
    // Act and assert
    Assert.ThrowsException<System.ArgumentOutOfRangeException>(() => account.Debit(debitAmount));
}

}
```

(Рис. 12)

Создадим метод теста для проверки правильного поведения в случае, когда размер списания превышает баланс:
Присвоил debitAmount значение, превышающее баланс. (рис.13)

```
[TestMethod]
Ссылок: 0
public void Debit_WhenAmountIsMoreThanBalance_ShouldThrowArgumentOutOfRange()
{
    // Arrange
    double beginningBalance = 11.99;
    double debitAmount = 100.00;
    BankAccount account = new BankAccount("Mr. Bryan Walton", beginningBalance);
    // Act and assert
    Assert.ThrowsException<System.ArgumentOutOfRangeException>(() => account.Debit(debitAmount));
}

}
}
```

(Рис. 13)

Рефакторинг тестируемого кода

Сначала определяю две константы для сообщений об ошибках в области видимости класса. Добавляю это в тестируемый класс (BankAccount): (рис.14)

```
using System;
namespace BankAccountNS
{
    /// <summary>
    /// Bank account demo class.
    /// </summary>
    Ссылок: 10
    public class BankAccount
    {
        public const string DebitAmountExceedsBalanceMessage = "Debit amount exceeds balance";
        public const string DebitAmountLessThanZeroMessage = "Debit amount is less than zero";
        private readonly string m_customerName;
        private double m_balance;
        Ссылок: 0
        private BankAccount() { }
        Ссылок: 4 | 1/3 passing
        public BankAccount(string customerName, double balance)
        {
            m_customerName = customerName;
            m_balance = balance;
        }
```

(Рис. 14)

Затем изменяю два условных оператора в методе Debit: (рис.15)

```
Ссылок: 4 | 1/3 passing
public void Debit(double amount)
{
    if (amount > m_balance)
    {
        throw new System.ArgumentOutOfRangeException("amount", amount,DebitAmountExceedsBalanceMessage);
    }
    if (amount < 0)
    {
        throw new System.ArgumentOutOfRangeException("amount", amount,DebitAmountLessThanZeroMessage);
    }
    m_balance -= amount; // intentionally incorrect code
```

(Рис. 15)

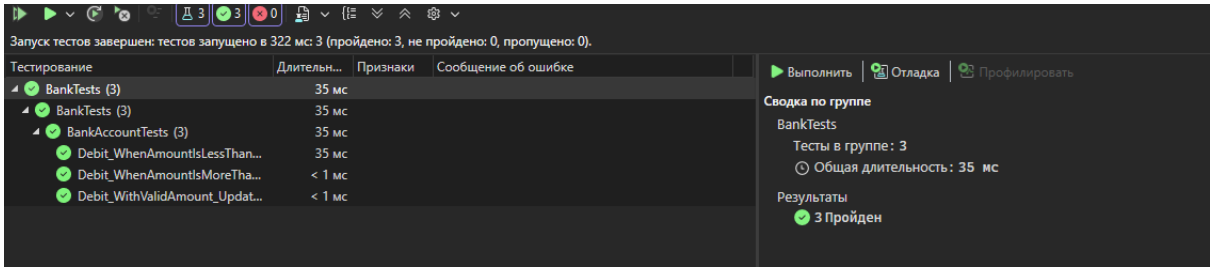
Рефакторинг тестовых методов

Выполняю рефакторинг методов теста, удалив вызов Assert.ThrowsException. Заключаю вызов Debit() в блок try/catch , перехватите конкретное ожидаемое исключение и проверив соответствующее ему сообщение. (рис.16)

```
[TestMethod]
Ссылок: 0
public void Debit_WhenAmountIsMoreThanBalance_ShouldThrowArgumentOutOfRange()
{
    // Arrange
    double beginningBalance = 11.99;
    double debitAmount = 20.0;
    BankAccount account = new BankAccount("Mr. Bryan Walton",beginningBalance);

    // Act
    try
    {
        account.Debit(debitAmount);
    }
    catch (System.ArgumentOutOfRangeException e)
    {
        // Assert
        StringAssert.Contains(e.Message,BankAccount.DebitAmountExceedsBalanceMessage);
```

(Рис. 16)

Заключительный проверочный запуск(рис.17)



(рис.17)

Вывод

Я освоил создание и запуск модульных тестов в Visual Studio на примере проекта банка. С помощью тестов я нашел ошибку в методе списания средств, исправил её и проверил результат повторным запуском. Также я научился улучшать код и тесты, чтобы они работали надежнее и эффективнее, а сообщения об ошибках стали подробнее.